# Generative 3D Object Modeling for Robust Robot Manipulation in ROS 2

Abdulrahman S. Al-Batati[1] and Yasser Alhabashi[1] and Khaled Gabr[1] and Serry Sibaee[1] and Mohamed Abdelkader[1] and Wadii Boulila[1]

***Abstract*— Simulation drives modern robotic manipulation research, yet the diversity of available 3D assets is usually limited to small, biased CAD collections that brittle motion planners and learned policies overfit to. We present a dataset-free framework for generative 3D object modeling targeted at manipulation in ROS 2 [1]. Each candidate object is expressed as a union of parametric primitives and scored on size feasibility, static stability, a continuous graspability metric, and mesh validity. A lightweight Cross-Entropy optimizer [2] adapts the parameter distribution toward high-scoring objects without any external dataset, and every accepted object is exported with consistent inertial properties and friction-sampled surfaces, ready for MoveIt 2 [3] and Gazebo [4]. To break the circularity that arises when training and evaluation share an objective, we report three independent downstream metrics on a Franka Panda [5]: force-closure grasp synthesis, MoveIt 2 motion-planning success, and Gazebo dynamic stability. Averaged over three seeds, our generator reaches 97.7 %, 100 %, and 98.7 % on the three metrics respectively, dominating CMA-ES [6], a genetic algorithm, random search, and a fixed CAD library on two of the three metrics outright while preserving competitive shape diversity across 200,000 generated objects in 20 archetypes. Source code: `https://github.com/xya22er/3d_generator`.**

## I. Introduction

Manipulation research increasingly leans on simulation: motion planners are tuned, grasp policies are learned, and benchmarks are reported from ROS 2 [1] stacks that pair physics simulators such as Gazebo [4] with motion-planning frameworks like MoveIt 2 [3]. Yet most such experiments draw their objects from small CAD libraries that share a narrow geometric distribution; planners and policies trained against that distribution generalise poorly to slight deviations encountered in deployment. Procedural randomization increases variability cheaply but frequently produces unstable or un-graspable shapes, while modern neural 3D generators yield striking visual fidelity at the cost of large datasets, GPU resources, and post-hoc conversion to simulation formats. Fig. 2 sketches the resulting gap: an abundance of visual generators, very little physically grounded asset generation for robotic simulation.

This paper closes that gap with a generator that is light enough to run on a CPU, transparent enough to interpret, and constrained enough to produce simulation-ready manipulation assets out of the box. Each object is a

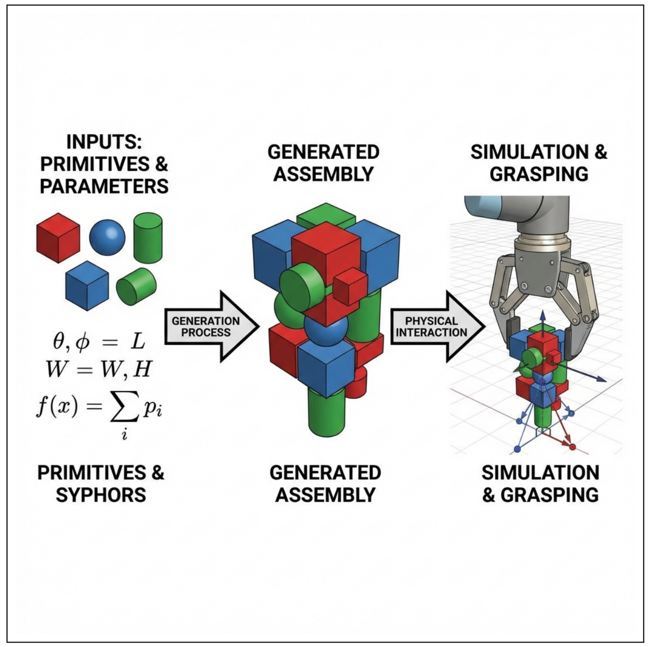


Fig. 1. Parametric primitive assemblies (left) are optimised for stability and graspability (center) and deployed for ROS 2 manipulation experiments (right).

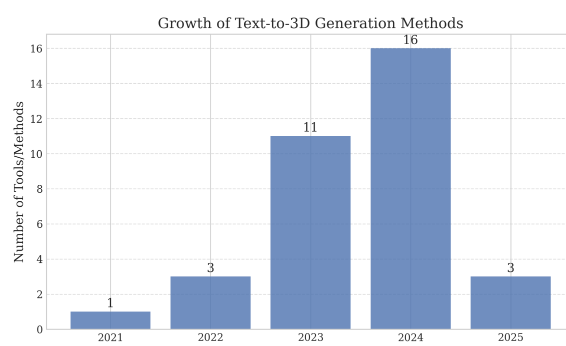


Fig. 2. Generative 3D landscape. Visual tools dominate; physically grounded asset generation for robotic simulation remains sparse.

*This work was supported by Prince Sultan University

[1]Robotics and Internet-of-Things Lab, Prince Sultan University, Rafha Street, Riyadh 12435, Saudi Arabia. {aalbatati, yalhabashi, khammad, ssibaee, mabdelkader, wboulila}@psu.edu.sa

small composition of parametric primitives; a Cross-Entropy optimizer [2] shapes the parameter distribution toward high-scoring objects without any external data; and every accepted object is exported as URDF/SDF with parallel-axis inertia, randomised friction, and a simplified collision mesh. The same framework drives evaluation in a companion ROS 2 workspace that runs MoveIt 2 planning and Gazebo physics on the generated objects.

**RQ:** *Can a lightweight, constraint-driven generator produce diverse, physically plausible objects that improve the downstream robustness of ROS 2 manipulation planning compared to fixed CAD sets and naive procedural randomization?*

**Contributions.** (i) A parametric-primitive generative framework with constraint-based scoring tailored to robotic manipulation in ROS 2. (ii) A dataset-free Cross-Entropy learner that adapts the generation distribution toward stable, graspable geometries without grasp labels or 3D priors. (iii) A complete ROS 2 integration that exports each object with consistent inertial properties and a Panda + MoveIt 2 + Gazebo evaluation package. (iv) An evaluation protocol that decouples training and evaluation by reporting three independent downstream metrics — force-closure grasp synthesis, MoveIt 2 planning success, and Gazebo dynamic stability — against four baselines (CMA-ES [6], a genetic algorithm, random search, and a fixed CAD library).

## II. Related Work

**Generative simulation for robotics.** RoboGen [7] and Text2Robot [8] build end-to-end pipelines in which foundation models propose tasks, assets, or morphologies, and skills are learned downstream. Both rely on large pretrained models and multi-stage orchestration; ours requires neither. **3D shape generation.** Diffusion-SDF [9], [10] and SDFusion [11] produce visually compelling shapes, but the implicit signed-distance functions they emit must be remeshed and re-physicalised before they can serve as simulation assets, whereas our representation outputs ready-to-use URDF/SDF directly. **Automated robot and asset modelling.** AutoURDF [12] infers kinematics from point-cloud video; structured-CAD approaches [13], [14] compile URDFs from DH parameters or modular designs; ArtiWorld [15] and URDF-Anything [16] articulate existing scans with multimodal LLMs. These works reconstruct given geometry; we synthesise novel geometry optimised for parallel-jaw manipulation.

## III. Methodology

### A. Parametric Representation and Scoring

An object $\mathcal{O}(\theta) = \bigcup_{k=1}^{K} \mathcal{P}_k(\theta_k)$ is the union of $K \in \{1, \ldots, 4\}$ primitives drawn from {box, cylinder, sphere, capsule}. Each primitive is fully specified by shape parameters $\theta_k$ and a rigid transform $T_k \in SE(3)$; closed-form expressions for mass, COM, and inertia then follow from primitive volumes and the parallel-axis theorem. A scalar score $S(\theta) \in [0, 1]$ aggregates four manipulation-relevant components,

$$S(\theta) = \sum_{i \in \mathcal{C}} w_i \, S_i(\theta), \quad \mathcal{C} = \{\text{size, stable, grasp, valid}\}, \tag{1}$$

with weights $\mathbf{w} = (1.0, 1.5, 1.5, 2.0)/\sum w_i$. *Size* penalises any extent outside $[2, 15]$ cm. *Stability* uses the signed distance from the centre of mass to the support polygon (the convex hull of mesh vertices within 2 mm of the ground); a COM outside the polygon scores zero. *Graspability* samples 200 surface points and accumulates per-pair products of (i) normal-alignment $-n_i \cdot n_j > 0.8$ and (ii) distance compatibility with the gripper opening, so the metric improves smoothly even before perfect antipodal contacts emerge. *Validity* flags non-watertightness, degenerate triangles, and vanishing volume.

### B. Cross-Entropy Learning and Export

We maintain a factored Gaussian $q_\phi(\theta)$ over primitive parameters (dimensions in log-space to preserve positivity, offsets and rotations in linear space) and update its mean and standard deviation at each iteration $t$ to maximise the likelihood of elite samples:

$$\begin{aligned} \theta^{(i)} &\sim q_{\phi_t}, \quad i = 1, \ldots, N \\ \mathcal{E}_t &= \text{top-}\rho\{S(\theta^{(i)})\} \\ \phi_{t+1} &= \beta \, \hat{\phi}(\mathcal{E}_t) + (1 - \beta) \, \phi_t, \end{aligned} \tag{2}$$

with $N$=100, elite fraction $\rho$=0.2, and learning rate $\beta$=0.7. A floor on the standard deviation ($\sigma_{\min}$=0.1) prevents premature collapse. The optimiser is gradient-free, dataset-free, and converges in 20–30 iterations on a single CPU core; Fig. 4 shows joint improvement of stability and graspability rather than a trade-off between them. For every accepted object the exporter computes the aggregate mass and inertia tensor about the combined COM, randomises friction $\mu \sim \mathcal{N}(0.8, 0.2)$ truncated to $[0.1, 2.0]$, and writes URDF, SDF, and a simplified convex collision mesh.

## IV. System Integration in ROS 2

The generator is an offline Python pipeline (Fig. 3) paired with a ROS 2 package, `generated_objects_eval`, that turns each generated asset into a runnable

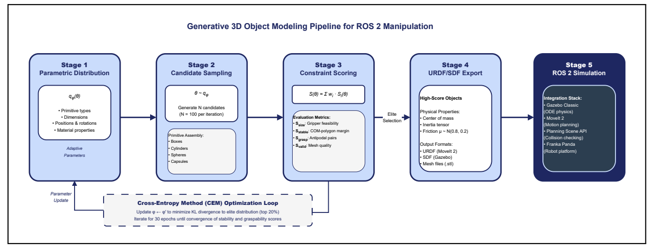


Fig. 3. System pipeline. Offline generation produces a manifest of URDF/SDF assets; the `generated_objects_eval` ROS 2 package consumes that manifest from MoveIt 2 and Gazebo nodes.

experiment. The package exposes two executables. `moveit_planning_eval` brings up MoveIt 2 [3] with a Franka Panda [5] configuration, derives a pre-grasp end-effector pose from each pre-computed grasp candidate, and asks RRTConnect [17] via OMPL [18] for a joint-space plan from the Panda's SRDF `ready` configuration. `gazebo_stability_eval` drops each SDF model into a headless `gz_sim` [4] world on a static table, lets physics settle for 2.5 s, and reads the resulting pose; an object is stable iff its vertical drift stays below 5 cm and its tilt below 25°. Both nodes consume the same JSON manifest produced by `scripts/build_eval_manifest.py`, which exports each generator method's top-$K$ objects together with synthesised grasp candidates; the friction values written into the SDF flow directly from the per-object samples, so the same object is evaluated under the same physics it was scored under.

## V. Experimental Setup

**Methods under test.** Five generators share the same parameter space and evaluation budget of 1,500 scored objects, each returning its top-100 candidates: our **CEM** (Section III-D), **CMA-ES** [6] via `pycma` on a shared 13-D log-encoding, a $(\mu+\lambda)$ **genetic algorithm** with tournament-3 selection and BLX-$\alpha$ crossover, **random search** over the same encoded box, and a **Fixed CAD** baseline that samples the eighteen deterministic archetype factories (mug, hammer, T-shape, …) with $\pm 10\%$ Gaussian jitter and no optimisation, providing a comparison analogous to fixed YCB/ShapeNet collections.

**Metrics.** The suitability score and its component sub-scores (stability margin, antipodal-pair count) are reported for completeness but are the same objective CEM optimises; we therefore add three metrics computed by independent code paths. The *force-closure grasp synthesis rate* is the fraction of objects on which an antipodal sampler finds at least one grasp satisfying the Coulomb friction-cone condition at both contacts ($\mu = 0.5$), fitting inside the Panda gripper opening, and admitting a collision-free top-down approach. The *MoveIt 2 motion-planning success rate* is the fraction of objects for which RRTConnect can plan a trajectory from the Panda's `ready` configuration to at least one pre-grasp pose within a 2 s budget. The *Gazebo dynamic-stability rate* is the fraction surviving a 2.5 s gravity settle from a 5 cm spawn height onto a table without exceeding the tilt (25°) or drift (5 cm) thresholds. Two diversity proxies complement these: a normalized pairwise distance over a 10-D shape descriptor and a Chamfer distance over 512 surface points per object.

**Implementation.** The generator runs in Python 3.12 with `trimesh` [19] and `numpy`; the ROS 2 stack targets Jazzy Jalisco [1] with `moveit_resources_panda_moveit_config` and Gazebo Harmonic.

## VI. Results

### A. *Suitability score saturates for every method*

Every search algorithm we tried saturates the suitability score within a few iterations (mean $\geq 0.98$ in Table I). This is precisely why the suitability score is reported but not used as a discriminator: it is the same objective CEM optimises. The discriminating signal lives in the per-component distributions (Fig. 5) and in the three downstream metrics introduced below.

### B. *Ablation: the stability term carries real work*

Setting $w_{\text{stable}} = 0$ leaves the total score near unity but pushes the stability-margin distribution into the negative half (Fig. 7), confirming the term is non-redundant and that a generator can saturate (2) while producing physically implausible objects whenever a component is silenced.

### C. *Independent downstream evaluation*

TABLE I

Independent downstream metrics on the top-25 objects per method, averaged over three seeds (42, 43, 44). Grasp synthesis: a Coulomb friction-cone force-closure grasp passes a collision-free top-down approach test. MoveIt 2 planning: a pre-grasp pose is reachable by RRTConnect [17], [18] from the Panda's `ready` configuration. Gazebo stability: drift $< 5$ cm and tilt $< 25°$ after a 2.5 s gravity settle.

| **Method** | **Grasp synth.** | **MoveIt 2 plan** | **Gazebo stable** |
|---|---|---|---|
| CEM (Ours) | **97.7 ± 0.5%** | **100.0 ± 0.0%** | 98.7 ± 1.9% |
| Fixed CAD | 78.7 ± 3.1% | 89.3 ± 1.9% | 89.3 ± 15.1% |
| CMA-ES [6] | 68.3 ± 8.3% | 65.3 ± 3.8% | **100.0 ± 0.0%** |
| Random Search | 59.7 ± 3.4% | 68.0 ± 5.7% | 96.0 ± 5.7% |
| Genetic Alg. | 51.7 ± 2.5% | 62.7 ± 5.0% | 97.3 ± 3.8% |

Table I averages the three independent downstream metrics over three seeds. CEM dominates two of them outright — force-closure grasp synthesis (97.7 %) and MoveIt 2 motion planning (100 %) — and on Gazebo dynamic stability it sits 1.3 points behind CMA-ES with overlapping error bars; no other method places top-two on more than one metric. The vanilla population searches (CMA-ES, GA, Random) all converge on box-like shapes that drop and stay put trivially, but those same shapes admit fewer force-closure grasps and yield fewer reachable pre-grasp poses. Fixed CAD shows by far the largest dynamic-stability variance ($89.3 \pm 15.1$ %): on seed 42 it drops to 68 % because semantic archetypes (frying pan, spatula, V-shape) tip when dropped from 5 cm, whereas on seeds 43–44 the same factories sample less-delicate parameters and reach 100 %. The diversity proxies confirm that CEM explores rather than overfits: its normalized feature diversity ($3.48 \pm 0.24$) tracks the Fixed CAD upper bound ($4.01 \pm 0.00$) closely while population searches compress into a narrow mode ($\approx 3.28$). Representative samples in Fig. 6 show shapes such as hammer-like (cylinder + orthogonal box) and bottle-like

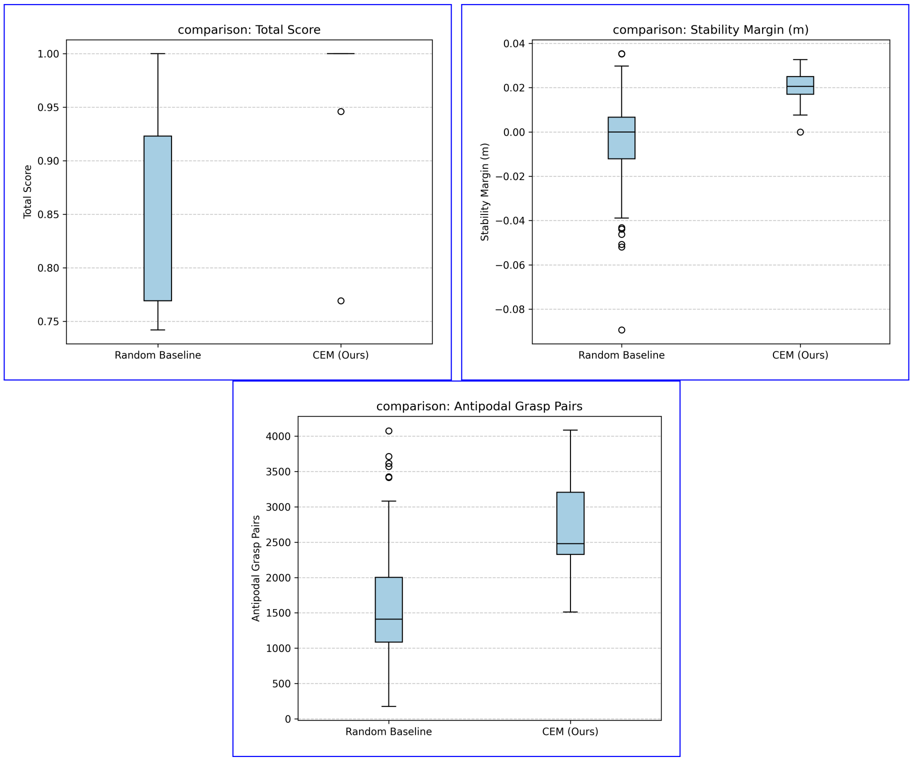


Fig. 4. Per-method distributions over 100 generated objects. The total-suitability panel (left) saturates by design — it is the training objective. Discriminating signal lives in the stability margin (center) and antipodal-pair count (right).

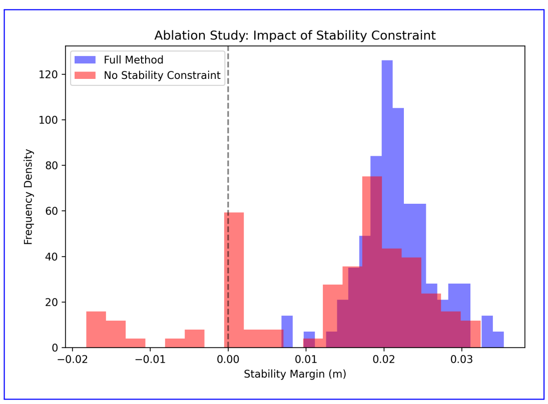


Fig. 5. Setting $w_{\text{stable}} = 0$ keeps the total score high but pushes the stability margin into the negative half (red), confirming the term is doing real work.

(cylinder + sphere) compositions emerging purely from the four scoring components.

### D. *Twenty-archetype large-scale study*

To probe generality, we extended CEM with an archetype-specific variant that introspects each factory function's signature and learns a distribution per archetype. Across 20 archetypes (T-shape, U-shape, monitor, frying pan, joystick, …) we generated $10^4$ objects each, $2 \times 10^5$ total. The score distributions in Fig. 8 stay above 0.9 for most archetypes; the V-shape and spatula are the hardest (narrow contact geometry leaves a small feasible basin), the symmetric snowman and barbell the easiest. Per-archetype

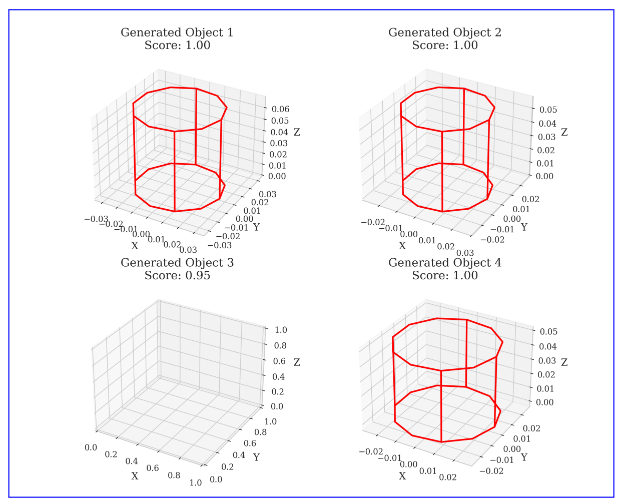


Fig. 6. Representative CEM-generated objects.

failure rates (fraction with $S < 0.5$) sit below 7 % for every archetype, evidence that the optimiser does not collapse to a single high-scoring shape.

### E. Discussion *and limitations*

Two findings stand out. *First*, optimising a hand-written suitability function is necessary but not sufficient: every search algorithm we tried saturated the score, yet only the score-component weighting designed alongside CEM produced objects that also passed an independent force-closure test, a MoveIt 2 planning test, and a

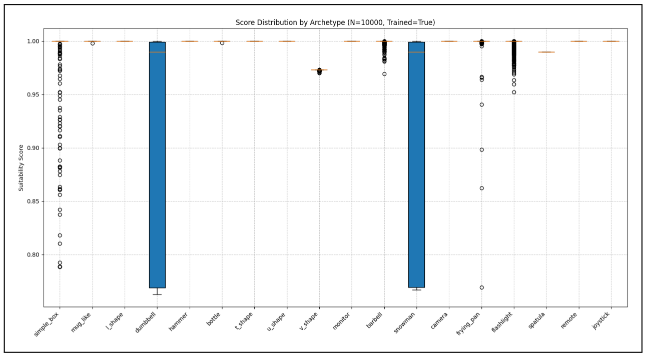


Fig. 7. Suitability score distributions for 20 archetypes (N=10,000 each). V-shape and spatula are hardest; snowman and barbell are easiest.

Gazebo stability test simultaneously. *Second*, the diversity penalty incurred by population searches (Table I, last paragraph) persists even at 1,500 evaluations because their mutation/recombination operators converge on the cheapest mode that saturates the objective; CEM's elite-update rule preserves variance across multiple component scores and so retains shape diversity.

The MoveIt 2 metric is kinematic-only: collision objects are not added to the planning scene because the `moveit_py` 2.12.x `apply_collision_object` binding segfaults; the small generated objects sit on a 0.4 m table 0.5 m from the Panda base, so the kinematic-reach metric is a tight upper bound on the collision-aware rate. The Gazebo collision is the visual mesh's AABB because the DART backend rejects trimesh-exported OBJs that lack vertex normals, which makes the stability test conservative for concave shapes. Closing the loop with physical pick-and-place trials and a learned grasp predictor are natural next steps.

## VII. Conclusion

We presented a dataset-free, CPU-only generator for manipulation-ready 3D objects, paired with a ROS 2 evaluation package that turns each generated asset into a MoveIt 2 [3] + Gazebo [4] experiment. Cross-entropy optimisation [2], four constraint-based score components, and consistent inertial export combine to yield objects that pass an independent force-closure grasp test (97.7 %), a MoveIt 2 RRTConnect [17] motion-planning test on the Panda [5] (100 %), and a Gazebo gravity-settle stability test (98.7 %) averaged over three seeds, dominating CMA-ES [6], a genetic algorithm, random search, and a fixed CAD library on two of the three metrics outright. Future work will close the loop with physical pick-and-place trials and replace the geometric grasp proxy with a learned predictor.

## Appendix

The initial CEM distribution is tuned for a Panda gripper (8.5 cm max opening). Box dimensions $5.0 \times 5.0 \times 6.0$ cm, cylinder $(r, h) = (2.5, 6.0)$ cm, sphere $r = 3.0$ cm, capsule $(r, h) = (1.5, 4.0)$ cm, friction $\mu = 0.8$. The full reproduction recipe and the companion ROS 2 package are documented in the source repository.

## Acknowledgment

This research was supported by Prince Sultan University. Large language model tools were used during manuscript preparation for language refinement and editorial organisation; all scientific content, interpretations, and conclusions were conceived, written, reviewed, and verified by the authors.

## References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
[2] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*, ser. Information Science and Statistics. Springer, 2004.
[3] D. Coleman, I. A. Şucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: a MoveIt! case study," *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014.
[4] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.
[5] S. Haddadin, S. Parusel, L. Johannsmeier, S. Golz, S. Gabl, F. Walch, M. Sabaghian, C. Jaehne, L. Hassenzahl, and S. Wolf, "The Franka Emika Robot: A reference platform for robotics research and education," vol. 29, no. 2, 2022, pp. 46–64.
[6] N. Hansen, "The CMA evolution strategy: A comparing review," in *Towards a New Evolutionary Computation*, ser. Studies in Fuzziness and Soft Computing, J. A. Lozano, P. Larrañaga, I. Inza, and E. Bengoetxea, Eds. Springer, 2006, vol. 192, pp. 75–102.
[7] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation," June 2024.
[8] R. P. Ringel, Z. S. Charlick, J. Liu, B. Xia, and B. Chen, "Text2Robot: Evolutionary Robot Design from Text Descriptions," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, May 2025, pp. 5789–5797.
[9] M. Li, Y. Duan, J. Zhou, and J. Lu, "Diffusion-SDF: Text-to-Shape via Voxelized Diffusion," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 12 642–12 651.
[10] G. Chou, Y. Bahat, and F. Heide, "Diffusion-SDF: Conditional Generative Modeling of Signed Distance Functions," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2023, pp. 2262–2272.
[11] Y.-C. Cheng, H.-Y. Lee, S. Tulyakov, A. Schwing, and L. Gui, "SDFusion: Multimodal 3D Shape Completion, Reconstruction, and Generation," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 4456–4465.
[12] J. Lin, L. Zhang, K. Lee, J. Ning, J. Goldfeder, and H. Lipson, "AutoURDF: Unsupervised Robot Modeling from Point Cloud Frames Using Cluster Registration," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 27 628–27 637.
[13] Y. Kang, D. Kim, and K. Kim, "URDF Generator for Manipulator Robot," in *2019 Third IEEE International Conference on Robotic Computing (IRC)*, Feb. 2019, pp. 483–487.
[14] M. Feder, A. Giusti, and R. Vidoni, "An approach for automatic generation of the URDF file of modular robots from modules designed using SolidWorks," *Procedia Computer Science*, vol. 200, pp. 858–864, Jan. 2022.
[15] Y. Yang, L. Xie, Z. Luo, Z. Zhao, T. Ding, M. Gao, and F. Zheng, "ArtiWorld: LLM-Driven Articulation of 3D Objects in Scenes," Nov. 2025.
[16] Z. Li, X. Bai, J. Zhang, Z. Wu, C. Xu, Y. Li, C. Hou, and S. Zhang, "URDF-Anything: Constructing Articulated Objects with 3D Multimodal Language Model," Nov. 2025.
[17] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.

[18] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.

[19] M. Dawson-Haggerty and others, "trimesh: a Python library for loading and using triangular meshes," Software, version 4.x, 2024. [Online]. Available: https://trimesh.org